

Chapter 10: Alternative Platforms

Gregory S. DeLozier, PhD

Our Pets application has two jobs: present an interface and preserve records. We have already separated those jobs with a database layer. Now we can change the interface without changing what a pet or an owner means.

This chapter has two parts. Part One builds the application with Streamlit. Part Two will build it with NiceGUI. Each platform has its own application and slide presentation. We begin here with Streamlit.

Part One: The Pets App in Streamlit

Hello World: A Button and a Counter

Start with one page, one button, and a number. From the course repository root, set up the environment and launch the example:

```
cd chapter-10-alternative-platforms/code/streamlit
python3 -m venv ~/venvs/pets-streamlit
source ~/venvs/pets-streamlit/bin/activate
python -m pip install -r requirements.txt
streamlit run hello_world.py
```

On Windows, create and activate the environment from PowerShell instead:

```
py -m venv "$HOME\venvs\pets-streamlit"
& "$HOME\venvs\pets-streamlit\Scripts\Activate.ps1"
```

Then run the same pip and Streamlit commands. Open the address printed in the terminal, normally `http://localhost:8501`. In a Codespace, open the forwarded port.

Here is the complete `hello_world.py` program:

```
import streamlit as st

st.title("Hello World")

if "count" not in st.session_state:
    st.session_state["count"] = 0

if st.button("Increment"):
    st.session_state["count"] += 1

st.write("Count:", st.session_state["count"])
```

The first run displays Hello World, an Increment button, and Count: 0. Click the button. Streamlit runs the script again, and `st.button("Increment")` returns True for that click. The program increments the count before displaying it. (1)

A plain `count = 0` near the top would reset the counter on every run. `st.session_state` keeps the value between runs for this browser session. The `if "count" not in ...` check initializes it only when it is absent. (2)

Try This: Click three times. You should see 3. Open the app in another browser tab; that session starts with its own count of 0. Session State is useful for the interface. The Pets application will use a database for records that must survive beyond a browser session.

Stop the counter server with Ctrl+C when you are ready to run the Pets application.

What Changes?

A web framework supplies machinery for building a browser-based application. (3) In our Flask version, we wrote routes and templates. Streamlit lets us describe the page with Python calls: a title, a table, a text field, or a button. Its normal execution model reruns the script when the user interacts with a widget. The script reads the current values and produces the page again. (1)

The database layer still offers familiar operations such as `get_pets()`, `create_pet(data)`, and `update_pet(id, data)`. Our Streamlit application calls those functions. It does not contain Structured Query Language (SQL) statements.

Responsibility	Flask example	Streamlit example
Describe the page	Templates	Python calls such as <code>st.dataframe()</code>
Receive input	Form data in a route	Widget return values
Choose an operation	Request route and method	Sidebar selection and submit button
Save records	Database functions	Database functions
Show the result	Redirect and render	Rerun and read the database again

Run the Application

Try This: In the same `code/streamlit` directory and activated environment:

```
python database.py
streamlit run app.py
```

Open the address printed by Streamlit. `streamlit run` starts the application server; the Python program runs on that server while the browser displays the interface. (4)

python database.py adds Alex, Jordan, Casey, and Luna to an empty database. Running it again keeps existing records. The application also works without the sample data: add an owner first, then a pet.

File	Purpose
hello_world.py	Counter and button introduction
app.py	Streamlit interface and actions
database.py	Tables, validation, and record operations
requirements.txt	Streamlit dependency
test_app.py	Database and form-interaction tests
data/pets.db	Your saved records

The database is stored beside these programs in data/pets.db. The repository ignores data/ folders. An optional PETS_DATABASE environment variable selects another file; the tests use it to keep their records separate.

The Page Is a Python Program

The application begins with:

```
import streamlit as st
import database

st.set_page_config(page_title="Pets", layout="wide")
database.initialize()
st.title("Pets and Owners")
```

initialize() creates the tables if they do not exist. It does not clear them or insert sample records during a rerun.

The sidebar provides the same choices we had in the earlier application:

```
page = st.sidebar.radio("Manage", ["Pets", "Owners"])
action = st.sidebar.radio("Action", ["List", "Add", "Edit", "Delete"])
records = database.get_pets() if page == "Pets" else database.get_owners()
st.header(page)
```

Selecting Owners makes the next run read owners. Selecting Pets makes it read pets. The result is a list of dictionaries, which we pass to a table:

```
if records:
    st.dataframe(records, hide_index=True, width="stretch")
else:
    st.info("No records yet.")
```

These are excerpts from app.py; run the supplied file rather than assembling a second copy from the chapter.

Forms Collect One Change

Adding a pet takes several values. We want to collect them together before writing a record. A Streamlit form batches its widget values and sends them when its submit button is pressed. (5)

In `pet_form()`, we read the owners, establish initial values, and create the form. Here is the name field and submission step:

```
with st.form(prefix):
    name = st.text_input("Pet name", value=pet["name"], key=prefix + "_name")
    # Age, type, food, and owner widgets are in this same form.
    submitted = st.form_submit_button("Save pet")
```

After the form, `if submitted:` controls the write. Ordinary page reruns read data; they do not insert another pet. The form gathers age with a nonnegative number input and uses text inputs for type and food.

Owners appear by name in a selection box, but the widget returns the owner's identifier. `format_func` changes the displayed label without changing the selected value. (6)

```
owner_id = st.selectbox(
    "Owner", owner_ids,
    index=owner_ids.index(current),
    format_func=lambda id: owner_names[id] + " (#" + str(id) + ")",
    key=prefix + "_owner",
)
```

Including the identifier in the displayed label also distinguishes two owners with the same name. The form returns an integer `owner_id` for the database operation.

Save, Then Read Again

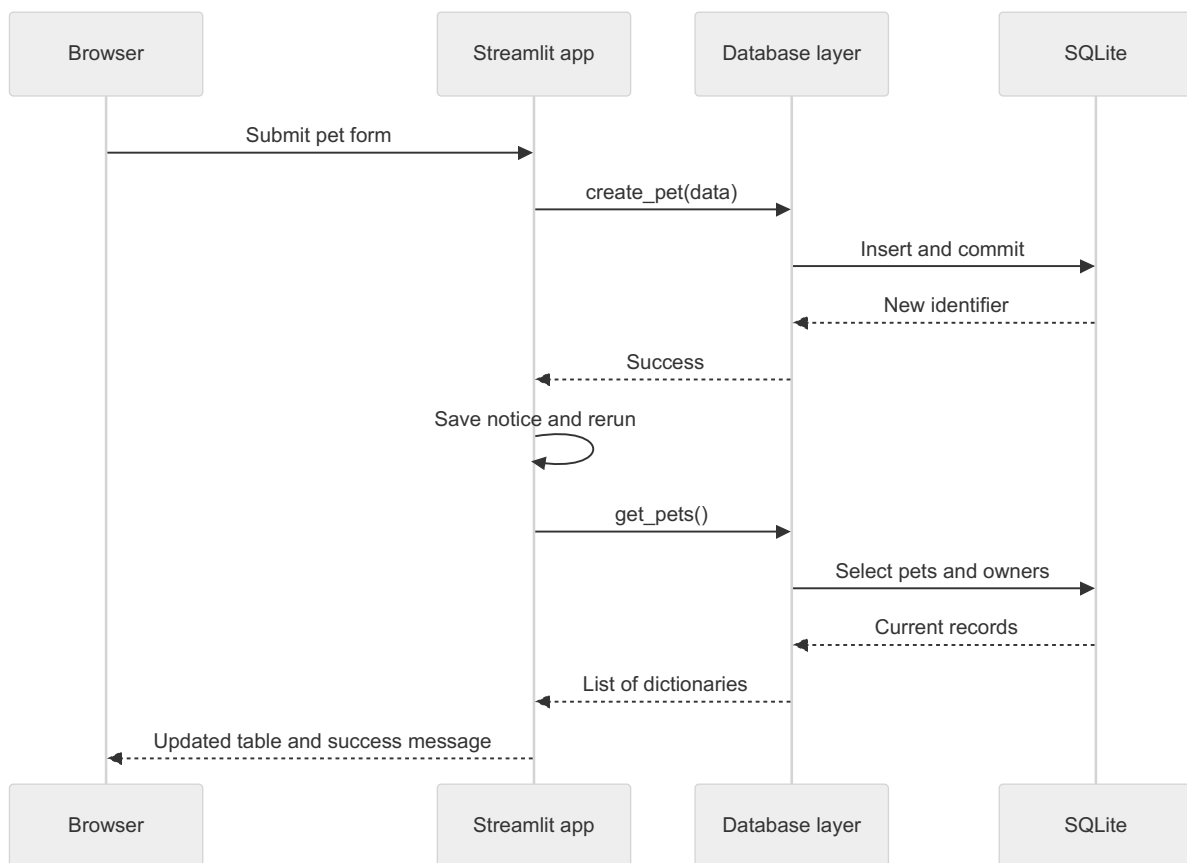
After submission, the program constructs the same kind of dictionary our database layer already accepts:

```
data = {"name": name, "age": age, "type": kind,
        "food": food, "owner_id": owner_id}
if "id" in pet:
    save(lambda: database.update_pet(pet["id"], data), "Pet updated.")
else:
    save(lambda: database.create_pet(data), "Pet added.")
```

The small `lambda` is a function that calls the database operation when `save()` invokes it. This lets one helper handle the common work: report an error or refresh after success.

```
def save(operation, message):
    try:
        operation()
    except ValueError as error:
        st.error(str(error))
    except database.ConstraintError:
        st.error("The change conflicts with a database rule. "
                "An owner with pets cannot be deleted; a pet needs an existing owner.")
    else:
        st.session_state["notice"] = message
        st.rerun()
```

`st.rerun()` stops the current run and schedules another. (7) The table was read before the form submitted, so rerunning makes it read the newly saved data. The next run does not submit the form again.



Page State and Saved Records

The success message must survive the rerun long enough to appear on the refreshed page. We store it in `st.session_state`, then consume it near the top of the script:

```
if "notice" in st.session_state:
    st.success(st.session_state.pop("notice"))
```

Session State keeps values between reruns for one browser session. (2) The notice belongs there. Pet records belong in SQLite, where they remain after the server stops.

State	Where it lives	Example
Local calculation	Current script run	records returned by a query
Interface state	Browser session's Streamlit state	Selected action, form values, notice
Persistent application state	SQLite file	Pets, owners, and their relationship

A second browser session has its own interface choices but reads the same database file. It sees another session's changes when it next reruns and queries the database.

Edit an Existing Pet

For Edit, the page selects a record by its identifier and passes that record to `pet_form(record)`. The form uses its stored values as defaults. On submission it calls `update_pet()` instead of `create_pet()`.

The widget keys include the pet's identifier:

```
prefix = "pet_" + str(pet.get("id", "new"))
```

That gives Casey's name field a different key from Luna's. Switching records loads the selected pet's form instead of treating all edit fields as one widget. The Add form uses `pet_new`.

Try This: Change Casey's owner to Jordan. Save, select List, then open Casey for editing again. The owner selection and the table should agree. Restart the Streamlit server and check once more.

Owners and Deletion

The Owners page follows the same pattern, using name, city, and type of home. Owners can be added and edited independently of pets.

Delete requires a checked confirmation for the selected record. Its key includes the record identifier, so confirming one record does not confirm a different record selected afterward. The database also enforces the relationship: deleting an owner who still has pets fails. Reassign or delete those pets first.

Each database operation opens its own connection, enables foreign keys, and closes the connection when finished. Successful writes commit; failed writes roll back. This keeps the connection's lifetime inside the database layer while Streamlit manages the interface.

Exercise: Follow a Record Through the Interface

Create a new owner and add two pets for that owner. Edit one pet's food and reassign the other pet to Alex. Try deleting the new owner while one pet still belongs to that owner. Then reassign the remaining pet and repeat the deletion.

Open the application in a second browser tab. Make a change in the first tab, then cause the second tab to rerun by changing its sidebar selection. Compare the selected page with the saved records. Which state is shared?

Finally, add a pet-type filter above the table. Keep the original record identifiers when filtering. Decide whether to filter the returned dictionaries or add an optional argument to `get_pets()`. Explain what would change if the table contained a million pets.

Part Two: The Pets App in NiceGUI

The NiceGUI implementation and its separate slide presentation will follow. Part One is complete on its own.

References

1. Streamlit. “Basic concepts of Streamlit.” <https://docs.streamlit.io/get-started/fundamentals/main-concepts>
2. Streamlit. “Add statefulness to apps.” <https://docs.streamlit.io/develop/concepts/architecture/session-state>
3. Wikipedia contributors. “Web framework.” https://en.wikipedia.org/wiki/Web_framework
4. Streamlit. “Understanding Streamlit’s client-server architecture.” <https://docs.streamlit.io/develop/concepts/architecture/architecture>
5. Streamlit. “st.form.” <https://docs.streamlit.io/develop/api-reference/execution-flow/st.form>
6. Streamlit. “st.selectbox.” <https://docs.streamlit.io/develop/api-reference/widgets/st.selectbox>
7. Streamlit. “st.rerun.” <https://docs.streamlit.io/develop/api-reference/execution-flow/st.rerun>

Questions for Thought

1. Why is `create_pet()` inside a submission branch rather than at the top of the script? Try describing what would happen on three ordinary reruns.
2. What would change if the pet list lived only in Session State instead of SQLite? Consider another browser tab and a server restart.
3. Why does an owner selection need an identifier even when the interface displays a name?
4. What part of this implementation would you keep when building the NiceGUI version? What part belongs specifically to Streamlit?
5. Two people open the same pet for editing. One saves a change, then the other saves an older form. What behavior would you choose for that situation, and what additional information would the database layer need?